



Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Gradient Boosting from Residuals

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Gradient boosting: three moves

1. Start from the mean

Every bond's fitted value is the average log spread.

2. Fit a tree to residuals

What the fit leaves unexplained: target minus fit.

3. Add a fraction of it

Learning rate times the tree's output. Back to 2.

```
fitted = fitted + learning_rate * tree.fit(X, y - fitted).predict(X)
```

The benchmark: Course 4's 615 training bonds, target log_oas. Synthetic bonds; a fitted spread is never a fair value.

Round 0: start from the mean

4.8776

mean log spread

131.3 bp

after exp: a geometric mean

354.1335

squared residuals to shrink

```
f0 = y.mean()  
residual = y - f0
```

A residual is positive for a bond wider than the level and negative for a tighter one.

The first stump, by hand

The split that leaves the smallest squared residuals

The stump splits the rating scale between BBB and BBB-.

score <= 9.5

	Left (score <= 9.5)	Right
Bonds	327	288
Leaf value (mean residual, log)	-0.5401	0.6132
After round 1, learning rate 0.1	124.4 bp	139.6 bp

Round 0 had every bond at 131.3 bp. Round 1 moves each side a tenth of the way, in log units.

boost_by_hand: equal to the library within 1e-12

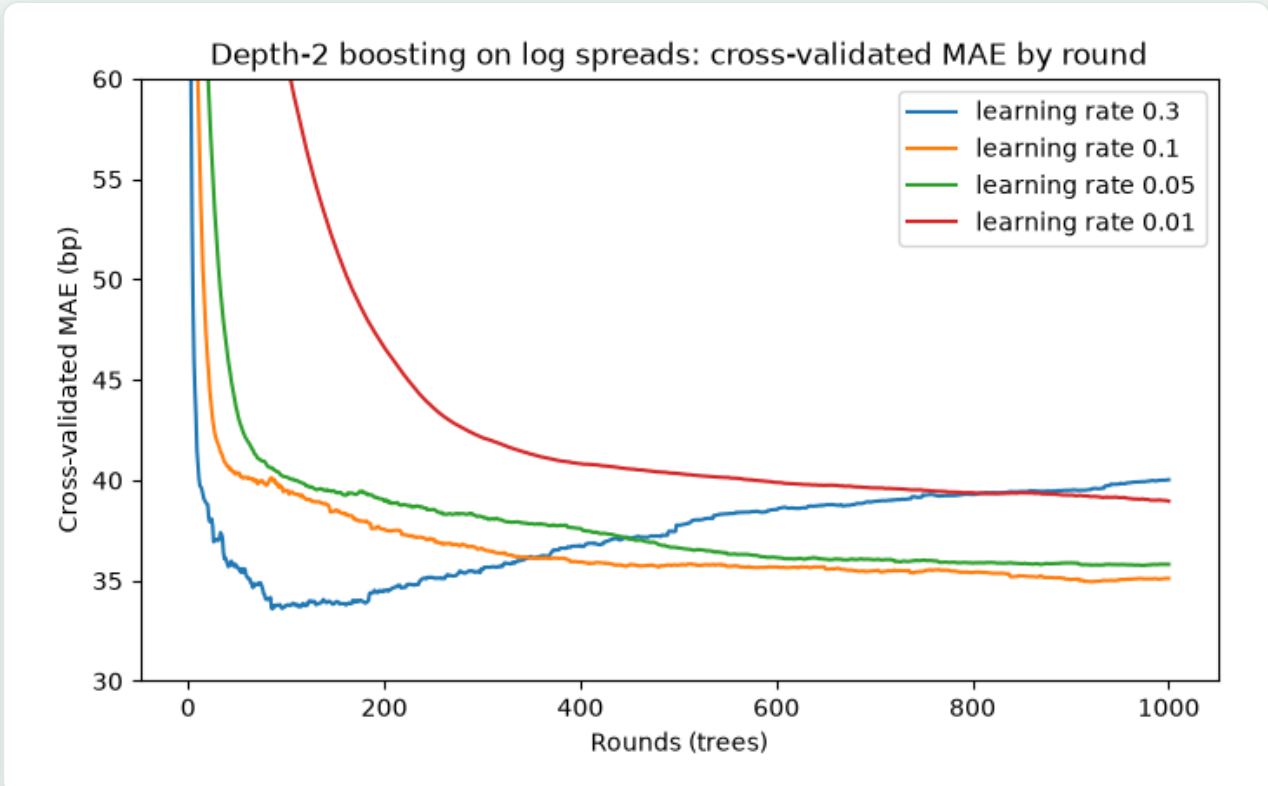
```
fitted = np.full(n, target.mean())
for _ in range(n_rounds):
    tree = DecisionTreeRegressor(...).fit(X, target - fitted)
    fitted = fitted + learning_rate * tree.predict(X)
```

Bond	oas (bp)	after 1	after 10	after 100
99130AAC6	49	118.4	75.6	53.8
99035JAB5	59	118.4	75.6	59.4
99143NAC3	22	118.4	61.2	33.7

100 rounds of depth-2 trees by hand equal GradientBoostingRegressor on all 615 bonds. 28 tests pass at the end of today.

Jerome H. Friedman, "Greedy function approximation: A gradient boosting machine.", The Annals of Statistics, vol. 29, issue 5, October 2001. Source, read 5 October 2026: the Crossref record (DOI 10.1214/aos/1013203451). Bibliographic facts only; the paper itself was not read for this course.

The learning rate against the number of rounds



Rate	Lowest	At round	At 300
0.30	33.6	86	35.6
0.10	34.9	923	36.6
0.05	35.8	973	38.2
0.01	39.0	1000	42.1

These curves describe; they do not choose. The comparison uses one setting fixed before looking: 300 rounds at 0.05.

Depth: one feature at a time, or two together

300 rounds, score and years	2 to 20 years, score 5	score 12	difference	trees on both
Stumps	0.2179	0.2179	0.0000	0
Depth 2	0.2126	0.2606	0.0481	160

Cross-validated MAE (bp), 300 rounds	Stumps	Depth 2
Score and years	39.9	40.7
Score, duration, and years	40.0	36.2

In logs the generator adds a rating term and a maturity term: no interaction, and stumps win. Duration is computed from the yield, which includes the spread, and with it depth 2 wins.

Boosting beside Course 4's OLS: cross-validated MAE in bp

Features	OLS (Course 4)	Boosting, stumps	Boosting, depth 2
Without coupon	41.7	40.0	38.2
With coupon	37.2	37.2	37.1

Coupon partly restates the spread (Course 4). With it, what is left for trees to find is small.

Allowed names

fitted spread

the model's spread for these characteristics

residual

Never called

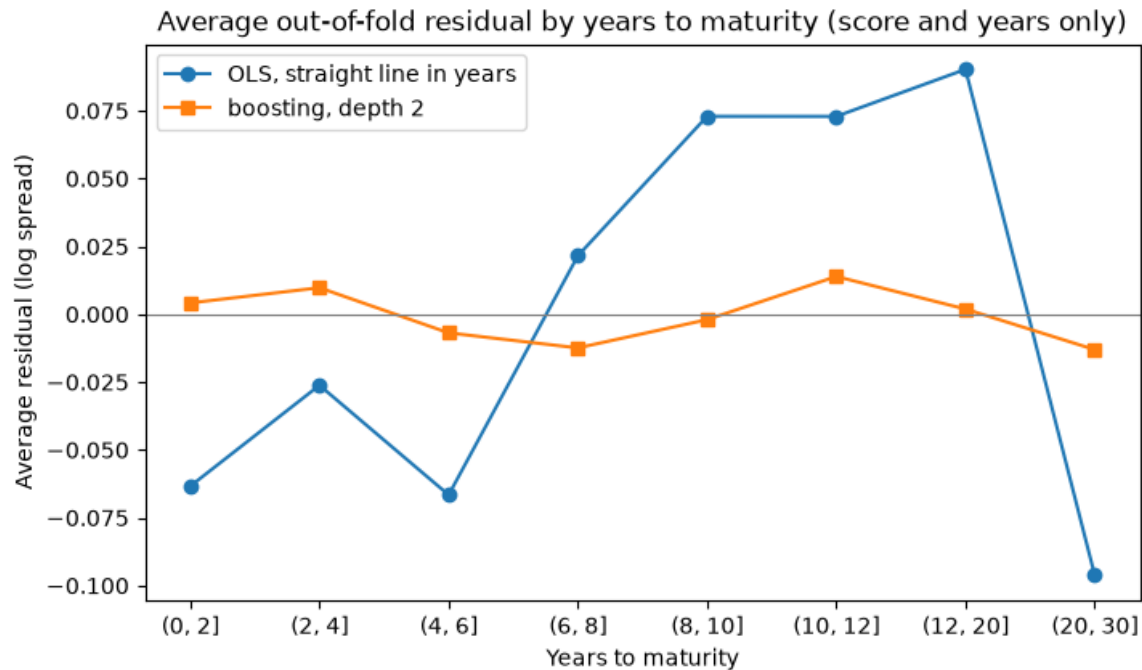
fair value or fair spread

cheap, rich, mispriced

a signal or a trade

Where the gain comes from: the generator's cap at 10 years

```
oas = BASE_OAS[score] * (0.70 + 0.30 * min(years_to_maturity, 10) / 10) * ...
```



42.3

OLS, bp

40.7

depth 2, bp

39.9

stumps, bp

Score and years only. A straight line in years cannot bend at 10; one tree question, years ≤ 10 , can. Day 12 writes the cap as a feature.

Two names for later

GradientBoostingClassifier

Start at the log-odds of the default rate; fit each tree to outcome minus probability, the gradient; add a fraction to the log-odds. XGBoost and LightGBM (days 8 and 9) build on it.

HistGradientBoostingRegressor

Columns in at most 255 bins; only the bin edges are tried. Trees grown by a number of leaves; missing values taken as they are. Named today, not fitted.

Excel side by side: one round from residuals

Excel side by side (tested in Excel)

`=AVERAGE(B2:B616)`

`=B2 - E2`

`=AVERAGEIFS(F2:F616,A2:A616,"<="&M2)`

`=AVERAGEIFS(F2:F616,A2:A616,">"&M2)`

`=E2+$M$1*IF(A2<=$M$2,$M$5,$M$6)`

`=AVERAGE(ABS(EXP(B2:B616) - EXP(D2:D616)))`

round 0: 4.8776

residual, column F

left leaf -0.5401

right leaf 0.6132

round 1, column G

MAE 38.2 bp

No gradient boosting function in Excel. Round 2 repeats the three steps; both rounds equal the library on all 615 bonds.

Save the day: git log -p -1 -- ensemblekit.py

1. Morning

Files as day 6 left them

2. Code

boost_by_hand and mae_bp

3. Results

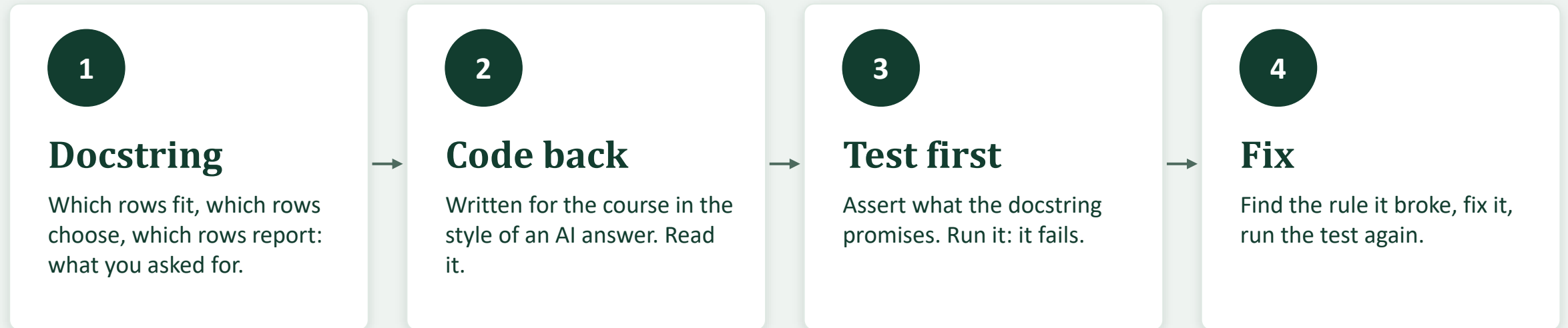
results.csv, cross-validated

```
model,features,rows,mae_bp,r2
OLS,rating_sector_duration_years_amount,cv,41.7166,0.8799
boosting_stumps,score_sector_duration_years_amount,cv,40.0452,0.8871
boosting_depth_2,score_sector_duration_years_amount,cv,38.1729,0.8999
```

The patch skips the results commit and shows the code commit: + lines only, adding def boost_by_hand and def mae_bp.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.



Today: each tree is fitted to y , not the residual: 100 rounds, 4 distinct fitted values.

Code that runs is not the same as code that is right.

What you can do now

- Each round: a small tree on the residuals, a fraction added
- A regression stump is a search for the smallest squared residuals
- `boost_by_hand` equals `GradientBoostingRegressor` to $1e-12$
- A learning-rate chart describes; it does not choose
- Stumps add one feature at a time; depth 2 adds interactions

Week 2, Day 3 XGBoost, regularization, and early stopping